

D'après tes documents, les plus importants sont :

- `pathlib` (très utilisé pour fichiers et dossiers)
 - `argparse` (scripts avec arguments)
 - `shutil` (copie, compression)
 - `subprocess` (commandes système)
 - `datetime` et `time`
 - `os` (mais souvent remplacé par `pathlib`)
 - gestion des exceptions (`try/except` , `raise` , `assert`)
-

Entraînement type examen (niveau réel)

Exercice 1 — Classe + fichier (CSV + pathlib)

Sujet

Créer un programme qui :

- Utilise `pathlib` pour vérifier si le fichier `machines.csv` existe
 - Crée une classe `Machine`
 - Lit le fichier CSV
 - Stocke les objets dans une liste
 - Ajoute une méthode `__str__` pour affichage
 - Permet de rechercher une machine par IP
-

Solution

```
from pathlib import Path

class Machine:
    def __init__(self, id: int, nom: str, adresse_ip: str):
        self.id = id
        self.nom = nom
        self.adresse_ip = adresse_ip

    def __str__(self):
        return f"{self.id} - {self.nom} - {self.adresse_ip}"

def lire_machines(fichier):
    chemin = Path(fichier)

    if not chemin.exists():
        print("Fichier introuvable")
```

```

    return []

machines = []

with chemin.open("r") as f:
    lignes = f.readlines()

for ligne in lignes[1:]:
    id, nom, ip = ligne.strip().split(",")
    machines.append(Machine(int(id), nom, ip))

return machines

def rechercher_machine(machines, ip):
    for m in machines:
        if m.adresse_ip == ip:
            return m
    return None

machines = lire_machines("machines.csv")

ip = input("IP à rechercher : ")
resultat = rechercher_machine(machines, ip)

if resultat:
    print(resultat)
else:
    print("Introuvable")

```

Exercice 2 — Fichier + argparse + pathlib

Sujet

Créer un script qui :

- Prend un dossier en argument avec `argparse`
 - Parcourt récursivement les fichiers
 - Calcule :
 - nombre de fichiers
 - taille totale (en KO)
 - plus gros fichier
-

Solution

```
import argparse
from pathlib import Path

parser = argparse.ArgumentParser(description="Analyse de dossier")
parser.add_argument("dossier", type=str, help="Chemin du dossier")

args = parser.parse_args()

base = Path(args.dossier)

total_fichiers = 0
taille_totale = 0
plus_gros = ("", 0)

for fichier in base.rglob("*"):
    if fichier.is_file():
        total_fichiers += 1
        taille = fichier.stat().st_size
        taille_totale += taille

        if taille > plus_gros[1]:
            plus_gros = (fichier.name, taille)

print("Nombre de fichiers :", total_fichiers)
print("Taille totale (K0) :", taille_totale / 1024)
print("Plus gros fichier :", plus_gros)
```

Exercice 3 — Fichier + suivi en temps réel (type log)

Sujet

Créer un script qui :

- Surveille un fichier texte
- Affiche les nouvelles lignes ajoutées en temps réel
- Utilise `time.sleep()`

Solution

```
import time

def suivre_fichier(fichier):
    with open(fichier, "r") as f:
        f.seek(0, 2) # aller à la fin du fichier

        while True:
```

```
        ligne = f.readline()

        if not ligne:
            time.sleep(1)
            continue

        print(ligne.strip())

suivre_fichier("log.txt")
```

Exercice 4 — Subprocess (très probable)

Sujet

Créer un script qui :

- Exécute `ipconfig` (Windows) ou `ifconfig` (Linux)
- Récupère la sortie
- Extrait l'adresse IP
- Affiche : `IP : xxx.xxx.xxx.xxx`

Solution

```
import subprocess
import platform

def obtenir_ip():
    systeme = platform.system()

    if systeme == "Windows":
        cmd = "ipconfig"
    else:
        cmd = "ifconfig"

    result = subprocess.run(cmd, capture_output=True, text=True, shell=True)

    lignes = result.stdout.split("\n")

    for ligne in lignes:
        if "IPv4" in ligne or "inet " in ligne:
            print("IP trouvée :", ligne.strip())
            break
```

```
obtenir_ip()
```

Exercice 5 — shutil (copie + zip)

Sujet

Créer un script qui :

- Copie un dossier `projets`
 - Le compresse en `.zip`
 - Supprime la copie
-

Solution

```
import shutil

source = "projets"
copie = "sauvegarde_projets"

shutil.copytree(source, copie)

shutil.make_archive("sauvegarde_projets", "zip", copie)

shutil.rmtree(copie)

print("Sauvegarde terminée")
```
